Assurance-Burden Mismatch: A Comparative Analysis of File-Based and Parametric Agent-Definition Architectures in Industrial Operational Technology

Pieter van Schalkwyk[1]
CEO, XMPro
ORCID: 0009-0009-3982-9346
pieter.vanschalkwyk@xmpro.com

Gavin Green[1]
Chief Solutions Officer, XMPro
ORCID: 0009-0007-8315-5559
gavin.green@xmpro.com

[1]XMPro Inc., Dallas, Texas, USA

April 2026

Executive Summary

The AI industry has converged on a new convention for defining how AI agents behave: directories of markdown files with embedded natural-language instructions, stored in code repositories like any other configuration file. Anthropic, OpenAI, GitHub, Cursor, and the Linux Foundation–stewarded AGENTS.md specification have all adopted this pattern in the last twelve months. It now governs the behavior of agents installed on developer workstations and, increasingly, on the operational technology systems that run industrial plants.

This whitepaper argues that the file-based convention, workable in horizontal developer tooling, is structurally mismatched with the assurance requirements of operational technology (OT). The case rests on three observations.

First, the file-based pattern is itself the primary attack surface. Independent research published in 2025–2026 documents attack success rates between 11.6% and 33.5% across production agent frameworks. Ecosystem audits found 13.4% of marketplace skills contain critical-severity issues. Four production Common Vulnerabilities and Exposures (CVE) entries were disclosed in twelve months, including one (CVE-2025-59536, CVSS 8.7) that triggers remote code execution simply by opening a cloned repository.

Second, six independent regulatory and standards frameworks governing industrial software — IEC 62443-4-1, NERC CIP-010, 21 CFR Part 11, GAMP 5, IEC 61511, and EU AI Act Annex IV — converge on the same four control objectives: unique identification, change control, status accounting, and audit. A markdown skill file satisfies none of these natively. Compensating controls can be retrofitted around the artifact, but in regulated industrial environments the cost of retrofit is regulated, not discretionary.

Third, parametric agent definitions — typed, schema-validated parameter manifests with deterministic content hashes — satisfy all four control objectives by construction. They align with the configuration primitives already established in OT: PI Points, IEC 61131-3 function

blocks, ISA-88 batch recipes, ISO 14224 equipment classes. Industrial AI deployments adopting this primitive sit naturally inside the systems they integrate with rather than fighting them.

The recommendation is structural. Original equipment manufacturers integrating AI agents into industrial offerings should specify parametric agent-definition contracts in tender requirements. Regulators should issue clarifying guidance applying existing configuration-management requirements to agent-definition artifacts. Practitioners deploying agents in or near regulated processes should treat agent definitions as configuration items under existing change-control regimes today.

The window for selecting an appropriate primitive — before the file-based pattern becomes the default architectural inheritance of industrial AI for the remainder of the decade — is narrowing.

Abstract

The contemporary architecture for defining agent behavior in large language model (LLM) coding and operations agents has converged on a file-based pattern: directories containing markdown files with YAML frontmatter, optional executable scripts, and free-form natural-language instructions. This pattern is now an industry convention, adopted across Anthropic Claude Code, OpenAI Codex, Cursor, GitHub Copilot, and the AGENTS.md cross-vendor specification.

Empirical evidence accumulated through 2025 and 2026 demonstrates that the file-based medium itself constitutes the primary attack surface in LLM-integrated agent systems. Qu et al. [39] tested 1,070 adversarial skill files against four production frameworks including Claude Code and reported attack success rates between 11.6 and 33.5 percent, alongside four disclosed production vulnerabilities. Snyk's *ToxicSkills* audit [40] found that 13.4 percent of 3,984 live skills in major marketplaces contained critical-severity issues. Documented production exploits — Pillar Security's *Rules File Backdoor* [35], Check Point's Claude Code vulnerabilities [7], PromptArmor's indirect prompt-injection demonstrations against Slack AI and Anthropic Cowork [36, 37], and Anthropic's own state-actor disclosure [4] — establish the attack class as production-grade.

This paper argues that the file-based pattern, while workable in horizontal developer-tooling contexts, is structurally mismatched with the assurance requirements of industrial operational technology (OT). Six independent regulatory and standards frameworks — IEC 62443-4-1, NERC CIP-010, 21 CFR Part 11, GAMP 5, IEC 61511, and EU AI Act Annex IV — converge on the same four control objectives: unique identification, change control, status accounting, and audit. Unstructured markdown satisfies none of these without retrofit. Parametric agent definitions — typed, schema-validated parameter manifests with deterministic content hashes — satisfy all four by construction.

The argument is structural and prospective: no production OT incident involving a file-based LLM skill is documented as of April 2026. The window for selecting an appropriate primitive before LLM agents migrate into safety-critical operational environments through original equipment manufacturer (OEM) integrations is narrowing.

Keywords: agent-definition architecture; operational technology security; IEC 62443; 21 CFR Part 11; prompt injection; supply-chain attacks; parametric configuration; LLM skills; agentic AI governance.

1 Introduction

The deployment of LLM-based agents in production environments has shifted from experimental tooling to industrial inflection within eighteen months. Anthropic's nation-state disclosure [4] describes a campaign in which Claude Code orchestrated 80 to 90 percent of operational decisions in a sustained cyber-espionage operation, with human operators retained only at four to six critical decision points. The same period has seen the emergence of a dominant architectural

pattern for defining agent behavior: file-based skill systems comprising markdown documents with YAML frontmatter, supporting scripts, and reference materials. Anthropic, OpenAI, Cursor, GitHub, and the Linux Foundation–stewarded AGENTS.md specification have converged on this primitive [2, 3, 34].

This convergence raises a question that the existing security literature has not directly addressed: what assurance properties does the file-based primitive offer in environments where assurance is regulated rather than discretionary? The applied security research on LLM agents — from Greshake et al. [15] on indirect prompt injection through to Qu et al. [39] on supply-chain poisoning — has been conducted predominantly against developer-tooling and information-technology (IT) targets. The operational-technology (OT) context, in which agent behavior intersects with safety-critical processes governed by IEC 62443, 21 CFR Part 11, NERC CIP, GAMP 5, IEC 61511, and OSHA Process Safety Management requirements, has received minimal direct treatment.

This paper provides that treatment. It argues that the file-based agent-definition pattern, irrespective of the substantial security engineering being layered onto it by individual harnesses (scanners, lockfiles, trust tiers, content-hash provenance), produces an assurance-burden profile that is structurally mismatched with OT compliance objectives. The argument has three parts. First, the file-based pattern is itself the primary attack surface, as evidenced by a coherent body of 2024–2026 research and disclosed exploits (sections 3 and 4). Second, the four control objectives that recur across every major OT compliance framework — unique identification, change control, status accounting, and audit — cannot be satisfied by unstructured markdown without substantial retrofit (section 6). Third, parametric agent definitions, defined as typed, schema-validated parameter manifests with deterministic content identity, satisfy these objectives by construction and align with established OT configuration primitives (section 7).

The paper concludes with explicit acknowledgement of the limits of the argument (section 8), the inflection point at which OEM-led OT integrations are now operating (section 9), and recommendations for OEMs, regulators, practitioners, and standards bodies (section 10). The argument is structural and prospective. As of April 2026, no production OT incident involving a file-based LLM skill is publicly documented. The thesis is not that file-based systems have failed in OT, but that the architectural class is being introduced into OT now, carrying with it a security and assurance profile that the OT regulatory framework was explicitly written to reject.

2 The Convergence on File-Based Agent Definitions

The file-based agent-definition primitive consists of a directory containing a markdown file (commonly named `SKILL.md`, `CLAUDE.md`, `AGENTS.md`, or vendor-specific equivalents) with YAML frontmatter declaring metadata such as name, description, version, dependencies, and platform constraints. The markdown body contains natural-language instructions for the agent. Optional subdirectories (`scripts/`, `references/`, `templates/`, `assets/`) hold supporting executable code, reference documentation, and assets that the agent may load on demand [1, 3].

Anthropic introduced Agent Skills in October 2025 [3] and declared the format an open standard in December 2025. OpenAI's Codex Skills [34] implements a functionally equivalent specification, with skills organized into three trust tiers: `.system` (auto-installed), `.curated` (installable by name), and `.experimental` (installable from arbitrary URLs). Willison [52] observed that the OpenAI implementation is functionally a clone of Anthropic's. The AGENTS.md specification, stewarded by the Linux Foundation, had been adopted in more than 60,000 GitHub repositories by January 2026 [16].

Cursor's `.cursor/rules/*.mdc` files extend the pattern with frontmatter `description` and `globs` fields. GitHub Copilot uses `.github/copilot-instructions.md`. Claude Code's `CLAUDE.md` is auto-injected into the context window at session start. In each case, the primitive is instruction-bearing markdown with light structural metadata, loaded into the agent's context either eagerly or on demand.

The standardization matters for the argument that follows. The convergence is sufficiently broad that whatever properties the primitive possesses or lacks become properties of the industry, not of a single vendor. Where this paper later argues that the primitive is mismatched with OT compliance, the argument applies across the field rather than to any single implementation.

Two structural features of the convention warrant explicit identification. First, the markdown body is unconstrained natural language; it may contain any instructions, in any tone, with any embedded references to external resources or executable commands. Second, the agent's behavior at runtime is determined by the natural-language instructions, interpreted by an LLM, rather than by a structurally validated specification. The harness may impose access controls and tool whitelists at the runtime layer, but the *definition* layer — what the agent is told to do — is unstructured by design.

3 Retrofitting Structure: Engineering Markdown Toward Safety

Because the medium of the agent definition is unstructured, the surrounding harness must reconstruct structural properties — content identity, change provenance, integrity verification, supply-chain trust — through external mechanisms. This section documents the mechanisms now in production or proposed, and argues that their existence constitutes evidence that the medium itself is insufficient.

Anthropic's published guidance on Agent Skills states explicitly: *"We strongly recommend using Skills only from trusted sources.... Malicious Skills could lead to data exfiltration, unauthorized system access... Audit thoroughly: review all files bundled in the Skill: SKILL.md, scripts, images, and other resources. Look for unusual patterns like unexpected network calls, file access patterns, or operations that don't match the Skill's stated purpose"* [3]. This audit checklist is functionally a delegation of security to the user. It is also a precise enumeration of the threat categories that a well-defined parametric specification would render structurally impossible: a parametric definition of a "PDF formatting" skill cannot contain a network call that does not appear in its declared capabilities, because the schema does not provide a slot for one.

OpenAI's three-tier trust hierarchy (`.system`, `.curated`, `.experimental`) addresses provenance through curation: skills installable from arbitrary GitHub URLs are placed in `.experimental`, with the implicit warning that users have assumed responsibility for review [34]. This addresses one dimension of the problem — distribution — without addressing the underlying issue that skills in any tier remain unstructured artifacts whose behavioral semantics are not machine-checkable.

The most explicit retrofitting effort documented in the public commentary is the Hermes Agent design described by Huang [18], which treats skills as a first-class harness subsystem. The published architecture includes: atomic file writes via `tempfile` and `os.replace` with full directory rollback if a security scan blocks installation; a regex-based pre-execution scanner of more than eighty patterns across nine threat categories (exfiltration via `curl` interpolating secret environment variables, prompt injection sequences, recursive root deletion, SSH backdoor installation, reverse shells via `/dev/tcp`, base64-decode-pipe-to-shell, NOPASSWD sudoers entries, embedded private keys, and zero-width or right-to-left Unicode used for textual obfuscation); a path-traversal-blocking validator that restricts supporting files to four allowed

subdirectories (`references/`, `templates/`, `scripts/`, `assets/`); an install policy mapping the Cartesian product of trust level (built-in, trusted, community, agent-created) and verdict (safe, caution, dangerous) to a three-way decision (allow, ask, block); and a `HubLockFile` providing a content-addressable provenance ledger that records, for each installed skill, source, identifier, trust level, scan verdict, SHA-256 content hash, install path, file list, and timestamps, with quarantine of skills whose hash mismatches on subsequent loads [18].

Snyk and Tessl have announced commercial offerings in this space [40], and Cisco AI Defense provides comparable scanning capabilities. The OWASP Agentic Skills Top 10 draft project [44] catalogues the threat classes that these scanners are designed to detect.

Three structural observations sharpen the argument. First, the SKILL.md frontmatter [18] is itself a partial parametric layer: typed fields for `name`, `description`, `version`, `platforms`, `prerequisites`, `required_environment_variables`, and `metadata` are validated by the harness at load time. The assurance failure is not in the typed frontmatter but in the unstructured prose body where the agent's actual operational instructions reside. The argument of this paper is therefore more precisely framed: the parametric layer needs to extend to the operational envelope, not stop at metadata.

Second, the Hermes function `_get_required_environment_variables()` reconciles three different syntactic forms in which a skill can declare secrets — `required_environment_variables`, `setup.collect_secrets`, and a legacy `prerequisites.env_vars` — into a single canonical list [18]. This is a concrete instance of the structural complexity tax that an unstructured medium imposes on its tooling: a typed schema with a single canonical secret-declaration field would render the reconciliation function unnecessary.

Third, Hermes implements a four-tier progressive-disclosure architecture for skill discovery (`skills_categories` returning category counts, `skills_list` returning name and description per skill, `skill_view` loading the full SKILL.md body, and `skill_view` with file path resolving individual supporting files on demand), with each `SKILL.md` read capped at the first 4,000 characters — enough for frontmatter [18]. The architecture exists because eagerly loading the unstructured prose body of every available skill is too expensive in tokens. A typed parametric definition does not require this hierarchy: schema-validated fields are cheap to enumerate without loading the full artifact.

Three further observations follow from the existence of this retrofitting effort. First, the engineering work is substantial and ongoing. Static pattern scanners, content-hash lockfiles, install-policy matrices, atomic-write-and-rollback systems, and quarantine pipelines are not minor adjuncts but distinct architectural layers added to make the primitive safe. Second, the threat categories the scanners are designed to detect are themselves the medium's failure modes: an artifact format that did not contain a slot for `curl ${SECRET} | sh` would not require a regex to detect that pattern. Third, the retrofitted properties — content identity (SHA-256 hashes), change provenance (lockfile records), integrity verification (hash-based tamper detection) — are intrinsic to a typed, hashable parametric definition. The file-based ecosystem is not engineering novel security properties; it is engineering toward properties that parametric configurations possess by construction.

4 Empirical Evidence: Demonstrated Vulnerabilities

This section documents the body of public research and disclosed incidents that establish the file-based primitive as a production attack surface. The evidence is drawn from peer-reviewed and preprint academic literature, vendor advisories with assigned Common Vulnerabilities and

Exposures (CVE) identifiers, security-firm responsible disclosures, and ecosystem-scale audits.

4.1 Foundational Research

Greshake et al. [15], in work presented at the AISec workshop, established the indirect prompt-injection threat class: retrieved content within an LLM's context can act as instructions to the model, with effects equivalent to arbitrary code execution. The taxonomy introduced — data theft, worming, ecosystem contamination, unauthorized API calls — has been corroborated and extended in subsequent literature on agentic systems [11]. The salience of the original finding for the present paper is that any artifact loaded into an LLM's context, including a skill file consulted to determine the agent's behavior, is a candidate for the attack class.

4.2 Production Exploits

Four publicly disclosed exploits in the period March 2025 to February 2026 establish the attack class as production-grade.

Pillar Security [35] disclosed the *Rules File Backdoor* attack in March 2025, demonstrating that hidden Unicode characters — zero-width joiners, bidirectional override markers — embedded in agent rules files for Cursor and GitHub Copilot can manipulate the agent into emitting backdoored code without visible cues to the human reviewer. Cursor and GitHub initially classified the issue as user responsibility for code review; GitHub deployed a user-interface warning for hidden Unicode on 1 May 2025 following public pressure [41].

Check Point Research [7] disclosed two vulnerabilities in Anthropic's Claude Code in February 2026. CVE-2025-59536 (CVSS 8.7) permits remote code execution through hooks injection in `.claude/settings.json` and `.mcp.json` configuration files, triggered by opening a cloned repository. CVE-2026-21852 (CVSS 5.3) permits API-key exfiltration through redirection of `ANTHROPIC_BASE_URL` to an attacker-controlled endpoint. Anthropic patched both vulnerabilities in January and February 2026 respectively. The exploit class is significant in the present context because the attack vector is a *configuration file*: opening a repository — a routine developer action — is sufficient to trigger code execution.

PromptArmor demonstrated indirect prompt injection against Slack AI in August 2024 [36, 45] and against Anthropic Cowork in early 2026 [19, 37]. The Cowork exploit is structurally significant: a malicious document posed as a Claude Skill instructed the agent to exfiltrate local files via the Anthropic Files API, which was on the agent virtual machine's outbound allowlist. The pattern demonstrates that the *trust* placed in skill-shaped artifacts is itself a vector — skills do not need to be malicious in their declared behavior to function as injection carriers.

Anthropic disclosed a state-actor cyber-espionage campaign in November 2025 [4]. The threat actor used Claude Code as an orchestration system, achieving 80–90 percent autonomy with human operators retained at four to six critical decision points per campaign. The disclosure is significant for the present argument not because it directly involves OT, but because it establishes that nation-state actors have integrated file-based-agent harnesses into multi-stage operational workflows.

4.3 Ecosystem-Scale Evidence

Two ecosystem-scale audits in early 2026 quantify the threat at marketplace scale.

Snyk's *ToxicSkills* audit [40] examined 3,984 skills across the major marketplaces. The audit

found that 534 (13.4 percent) contained at least one critical-severity issue, 1,467 (36.82 percent) contained at least one issue of any severity, and 76 contained confirmed malicious payloads, of which eight were live at the time of publication. The threat categories observed included credential exfiltration, malware distribution, and prompt-injection patterns.

Qu et al. [39], in a peer-reviewed empirical study, tested 1,070 adversarial skill files against four production frameworks including Claude Code. The study reported attack success rates between 11.6 percent and 33.5 percent across frameworks, and disclosed four production CVEs as a direct outcome of the research. The study is, to the present author's knowledge, the largest empirical evaluation of skill-file attack vectors in production frameworks to date.

Koi Security's *ClawHavoc* audit [23], published the same week as the Snyk audit, identified 335 malicious skills attributable to a single coordinated campaign within one marketplace. Antiy Labs [5] provided independent corroboration, cataloging 1,184 malicious skills tied to twelve publisher accounts and classifying the family as Trojan/OpenClaw.PolySkill.

4.4 Synthesis

The evidence base demonstrates three claims with high confidence. First, the file-based primitive is a production attack surface, not a theoretical one. Second, the attack class spans multiple vendors (Anthropic, OpenAI, Cursor, GitHub) and multiple delivery vectors (configuration files, skill files, rules files, document content). Third, the threat is observed at ecosystem scale, with double-digit prevalence rates in marketplace-distributed skills. One boundary of this evidence base: all documented incidents to date have occurred in developer-tooling and IT contexts. The OT extension of the threat class is the subject of section 6, but before reaching that argument, section 5 establishes that the IT and OT contexts are not minor variants of a common security model.

5 From IT to OT: An Inverted Threat Model

The empirical evidence in section 4 is drawn from information-technology (IT) and developer-tooling contexts. Before extending the argument to operational technology (OT), it is necessary to establish that the IT and OT contexts are not minor variants of a common security model but two materially different threat models with inverted priorities. The architectural choices made by horizontal AI vendors are calibrated for the first; the regulatory and operational consequences in scope for this paper apply in the second.

The Purdue inversion. IT security prioritizes confidentiality, integrity, and availability in that order. The Purdue Enterprise Reference Architecture [51] and IEC 62443 [20] jointly characterize OT security as inverting this ordering: availability and process safety are the primary objectives; confidentiality is subordinate. The inversion is not stylistic. A system unavailable for an hour in IT is an inconvenience; a system unavailable for an hour in a continuous process plant is a hydrocarbon release, a batch loss, a Safety Instrumented Function demand, or a grid frequency excursion.

Patch cadence asymmetry. The vulnerabilities documented in section 4 were patched on IT timescales: CVE-2025-59536 was patched by Anthropic in January 2026, days after Check Point disclosure [7]. The same patch applied to a 21 CFR Part 11–regulated environment must pass through documented change control, risk assessment, validation under GAMP 5 [21], and

pre-startup safety review under 29 CFR 1910.119(i). For NERC CIP-regulated devices, the patched version requires baseline reconfiguration and documentation under CIP-010-4 R1 [32]. For Safety Instrumented Systems under IEC 61511, full proof-test cycles may be required. The realistic patch interval for a critical-severity OT vulnerability is months to years, not days. In the interval, the unpatched version is the version running on the plant.

Lifecycle asymmetry. Information-technology systems operate on three- to five-year refresh cycles. Operational-technology systems operate on fifteen- to forty-year lifecycles, with installed bases including controllers commissioned in the 1990s still in continuous operation. An agent-definition primitive selected in 2026 will be inherited by OT systems whose operational life extends to 2046 and beyond. The architectural-inheritance horizon is not the next product release; it is a generation.

Identity infrastructure asymmetry. IT environments have largely converged on per-user identity with multi-factor authentication, single-sign-on, and per-action attribution. Control-room environments retain shared console sessions for shift continuity, vendor remote-access accounts with elevated and shared credentials, and engineering workstations frequently operated with administrator privileges. The SANS 2024 State of ICS/OT Cybersecurity Survey [9] documents that 25 percent of operators of industrial sites had not yet implemented multi-factor authentication for remote access by 2024, and that vendor and contractor access remains a recognized risk surface across the surveyed population. The 21 CFR Part 11 §11.10(e) requirement that audit trails record the identity of the operator who created or modified an electronic record [48] becomes structurally difficult, not merely procedurally inconvenient, in environments where shared session identity is normal practice. A parametric manifest with required parameter-level approver identity makes attribution architectural rather than aspirational.

Threat-actor inheritance. The state-actor disclosure in Anthropic [4] establishes that nation-state actors have integrated file-based agent harnesses into multi-stage operational workflows in IT and developer-tooling contexts. The actor class involved — nation-state, autonomous orchestration, four-to-six-decision human-in-the-loop — is the same actor class historically responsible for the documented OT cyber-physical incidents: Stuxnet against the Natanz uranium enrichment facility [25]; Industroyer/Crashoverride against the Ukrainian electrical grid [8]; TRITON/TRISIS against a Saudi petrochemical Safety Instrumented System [27]; and PIPEDREAM/Incontroller, the toolkit attributed to CHERNOVITE in 2022 and explicitly designed for industrial control system targeting [10]. Dragos's eighth annual OT cybersecurity year in review [12] documents continued growth in OT-targeting threat groups and an 87 percent year-over-year increase in OT-impacting ransomware in 2024. The combination of (a) demonstrated nation-state capability with file-based agent harnesses and (b) the same actor class's historical OT-targeting profile is not speculative; it is the explicit overlap of two documented capabilities now able to operate together.

Workstation file vector continuity. Stuxnet entered through compromised engineering workstations and propagated via malicious project files. TRITON entered via a workstation file vector, with `trilog.exe` masquerading as a legitimate Triconex log-reader. Both attacks exploited the trust placed in workstation-resident artifacts at the boundary between corporate IT and operational networks. Skill files distributed to engineering workstations through community marketplaces, vendor partner channels, or OEM integrations are the same architectural class: unverified or weakly-verified artifacts on engineering workstations whose modifications propagate into operational consequence. The empirical evidence in section 4 establishes the attack class

against this vector in IT; the historical OT incident base establishes the consequence class for the same vector in OT.

The consequence asymmetry. When an information-technology agent leaks a Slack token or exfiltrates source code, the consequence is data leakage: bad, recoverable, insurable. When an operational-technology agent misadvises an operator on a high-pressure separator, miscalibrates an alarm threshold, bypasses a permissive interlock, or recommends an erroneous setpoint, the consequence is in the cyber-physical domain: loss of containment, runaway exotherm, equipment destruction, or loss of life. The Buncefield (2005), Texas City (2005), Bhopal (1984), and Deepwater Horizon (2010) incidents — all involving failures at the boundary of human decision-making and process control — are the failure-mode reference class for this discussion, not the IT data-breach reference class.

The seven inversions above are not corollaries of the four-control-objective convergence in section 6; they are independent observations about the operating context. They establish that an architectural primitive optimized for one threat model cannot be transferred to the other without re-evaluation. The remainder of this paper takes that re-evaluation as its task.

6 The OT Compliance Gap

This section documents the structural mismatch between the file-based primitive and the assurance requirements of operational technology and regulated industrial environments. The argument proceeds from a meta-observation about the regulatory landscape, through specific clause-level requirements, to the consequences of attempting to satisfy these requirements with unstructured markdown.

6.1 The Four-Requirement Convergence

Six independent regulatory and standards frameworks impose configuration-management, audit, and change-control requirements on software components in regulated industrial contexts. They were authored by different organizations, in different jurisdictions, for different industries, across two decades. They converge on the same four control objectives.

The first is **unique identification**: every configuration item — whether software component, parameter set, document, or composite artifact — must possess a unique, persistent identifier that distinguishes it unambiguously from all other configuration items. The second is **change control**: no modification to a controlled configuration item may proceed without documented change request, impact assessment, qualified approval, and verification. The third is **status accounting**: the current configuration state of any controlled item must be knowable at any time, with a traceable record of changes. The fourth is **audit and review**: independent verification of configuration integrity, supported by secure, time-stamped, tamper-evident audit trails that do not obscure previously recorded information.

These four objectives are visible in IEC 62443-4-1's secure-product-development-lifecycle requirements [20], which inherit configuration-management discipline from IEEE 828; in NERC CIP-010-4 R1 [32], which mandates baseline configuration of operating system, firmware, application software, logical ports, and patches; in NIST SP 800-82 Rev 3's OT overlay of SP 800-53's Configuration Management family [30]; in 21 CFR Part 11 §11.10(e) and §11.10(k)(2) [48]; in GAMP 5 Second Edition's risk-based traceability matrix [21]; in IEC 61511's Safety Instrumented System lifecycle requirements; in OSHA's Process Safety Management requirements at 29 CFR

1910.119(l) [49]; and in EU AI Act Annex IV §6 and §9, which require description of relevant changes through the lifecycle and post-market monitoring [13].

The convergence is not coincidental. The four objectives are the minimum conditions under which an audit, an investigation, or a regulatory inspection can reconstruct what software was running, when, with whose authorization, and with what verified integrity.

6.2 What Markdown Cannot Provide Without Retrofit

A markdown skill file, considered as a configuration item, satisfies none of the four objectives natively.

It does not satisfy unique identification because the artifact has no canonical identity beyond its file path and name; identical content can exist at multiple paths, different content can exist at the same path across time, and there is no native cryptographic content hash binding identity to content. The Hermes Agent design [18], the OWASP draft [44], and Anthropic's recommended audit checklist [3] all introduce content hashing as an external retrofit precisely because the primitive lacks it.

It does not satisfy change control because modifications to a markdown file are not, by the artifact's structure, subject to any approval workflow, impact assessment, or qualified review. A user with file-system write access can modify the artifact at any time. Change control must be imposed by an external system — a Git workflow, a content-management system, an organizational policy — none of which is intrinsic to the primitive.

It does not satisfy status accounting because the natural-language body of a skill file does not produce semantically meaningful diffs. A change from "set the threshold to 0.7" to "set the threshold to 0.85" is, at the textual level, a small textual difference; a paragraph reordering is, at the textual level, a large textual difference that may carry no behavioral change. The system cannot determine, from the artifact alone, what changed in behavioral terms. This is the same problem 21 CFR Part 11 §11.10(e) was written to address: audit trails must record the substantive change, not merely that some change occurred.

It does not satisfy audit and review because there is no native, secure, computer-generated, time-stamped record of who authored a given version, who approved it, when it took effect, and which agent runs it governed. These properties can be recovered through external systems — an electronic document management system, a version-control history with signed commits, a quality management system — but the primitive itself does not carry them.

The retrofitting effort documented in section 3 — content-hash lockfiles, scanner pipelines, install-policy matrices — is precisely an attempt to provide these properties externally. The argument of this paper is that the retrofit is incomplete, costly, and unnecessary in environments where parametric definitions are an option.

6.3 Configuration Management in Specific Frameworks

IEC 62443-4-1 [20] requires four explicit configuration-management disciplines: configuration identification, change control, status accounting, and configuration audit. The standard inherits from IEEE 828 on configuration-management planning. SR 3.4 of IEC 62443-3-3 (Software and Information Integrity) mandates protection against unauthorized changes to code and configuration during operation; SR 7.6 mandates industry-best-practice configuration settings.

NERC CIP-010-4 R1 [32] requires Responsible Entities to implement documented processes

for configuration change management with baselines that include operating systems, firmware (where commercially available), commercially available or open-source application software intentionally installed, custom software intentionally installed, logical network accessible ports, and security patches. R1.2 requires authorization and documentation of changes that deviate from the baseline. R1.5 requires a testing environment that "models the baseline configuration." Implementation guidance contemplates per-device baselines for serial-connected devices — a parametric model.

NIST SP 800-82 Rev 3 [30] provides an OT overlay of SP 800-53 Rev 5, with the Configuration Management family (CM-1 through CM-14) directly applicable. CM-3 (Configuration Change Control), CM-6 (Configuration Settings), CM-8 (System Component Inventory), and CM-10 (Software Usage Restrictions) are particularly germane.

A markdown skill file, presented to an auditor under any of these frameworks, would require the operator to demonstrate that the artifact has been uniquely identified, that changes to it have been controlled and approved, that the current state is knowable and traceable, and that an audit trail records authorship and authorization. None of these properties is provided by the artifact itself. They must be provided by surrounding infrastructure that the operator has assembled. A parametric configuration item — a typed parameter set with a content hash, a registry record, and a workflow for approval — provides these properties as part of its definition.

6.4 Audit Trails and Electronic Records

21 CFR Part 11 §11.10(e) [48] requires "secure, computer-generated, time-stamped audit trails to independently record the date and time of operator entries and actions that create, modify, or delete electronic records." Critically, "record changes shall not obscure previously recorded information." §11.10(k)(2) requires revision and change-control procedures that maintain an audit trail documenting time-sequenced development and modification of systems documentation.

A change to a markdown skill file recorded in a Git history is not, in the regulatory sense, a Part 11–compliant audit trail. A Git history records commits and diffs; it does not record the *substantive* change in machine-checkable form. A regulator examining a Part 11 system would expect to be told, for any electronic record affecting a regulated process, when it changed, who made the change, who authorized it, and what specifically changed. For a parametric record, all four are expressible in semantic terms. For a markdown record, the fourth — what specifically changed in behavioral terms — is recoverable only by reading the prose.

GAMP 5 Second Edition [21] extends this requirement through risk-based computer system validation. The traceability matrix that GAMP 5 mandates — from user requirement specification, through risk assessment, to test design, to standard operating procedures and training — requires deterministic content identity for each artifact in the chain. A parametric manifest can be hashed and uniquely identified at every stage. A markdown skill can be hashed, but two skills that produce identical agent behavior can have different hashes (whitespace differences, paragraph reorderings), and two skills with different behaviours can have similar hashes if the changes are textually small.

FDA enforcement evidence in the period 2018–2025 illustrates the failure mode the requirements are designed to prevent. Warning letters to Kyowa Hakko Bio (2018), Case #622087 (2022), Westward Laboratories (2024), MMC Healthcare (2024), and Aspen Pharmacare (2025) repeatedly cite "lack of audit trails," reliance on "static printouts" rather than dynamic records, inability to attribute changes, and "insufficient audit-trail review." These are the failure modes of unstructured configuration artifacts being relied upon to satisfy structured audit requirements. Food and Drug Law Institute [14] report that 60–80 percent of FDA pharmaceutical warning

letters in recent years cite data-governance and data-integrity deficiencies under predicate rules 21 CFR 211.68 and 211.194.

6.5 Management of Change

OSHA 29 CFR 1910.119(l) [49] requires written procedures for any change — except "replacement in kind" — to process chemicals, technology, equipment, procedures, or facilities subject to Process Safety Management. The 2009 OSHA interpretation memo on Management of Organizational Change [49] extends the requirement to organizational, personnel, and policy changes that affect any of the five PSM elements.

IEC 61511 imposes equivalent requirements on Safety Instrumented Systems: bypass logs, proof tests, and revalidation cycles are required for any modification.

The connection to agent definitions is direct. An agent that influences a regulated process — even via advisory dashboards or operator-decision-support roles — falls within a defensible reading of "procedures" under 1910.119(l) and "computer or related systems" under 21 CFR 211.68(b). Modifying the agent's skill definition is a candidate trigger for Management of Change. A YAML-frontmatter markdown file changed via Git commit by any individual with repository access does not produce the documented impact assessment, training, pre-startup safety review, and qualified approvals that MOC requires. A parametric manifest change, routed through a change-control workflow with parameter-level approval gates, does.

6.6 Boundary of the Claim

As of April 2026, no public OT incident involving a file-based LLM skill as the primary attack vector has been documented in CISA, ICS-CERT, FBI Private Industry Notification, the National Transportation Safety Board, or the peer-reviewed literature. The argument of this paper is structural and prospective: the architectural class that has produced documented exploits in IT and developer tooling is the same class that is being introduced into OT through OEM integrations. The absence of OT incidents to date reflects the limited penetration of LLM agents into OT operational roles, not the absence of risk. The penetration is increasing.

7 The Parametric Alternative

A parametric agent definition is a typed, schema-validated specification of an agent's identity, capabilities, parameters, and operational envelope, identified by a deterministic content hash. The schema constrains what the definition may contain; the typing constrains how each field may be structured; the hash provides content identity independent of name or path. The runtime harness may load the definition and provide a natural-language explanation to the LLM, but the definition itself is data, not prose. This section defines the primitive, demonstrates its satisfaction of the four control objectives, and locates it within the established OT and software-architecture context.

7.1 Properties by Construction

A parametric definition satisfies unique identification because the content hash is the identifier, with the cryptographic property that two definitions with identical content produce identical

hashes and any modification produces a different hash. Two agents with the same parameter set and the same hash are the same agent.

It satisfies change control because the schema provides typed fields that can be subject to per-field approval policies. A change-control workflow can require sign-off on safety-related parameters specifically, or on changes that modify capability flags, without requiring blanket review of the whole definition.

It satisfies status accounting because the parameter values are semantic. A change from `pressure_threshold = 100` to `pressure_threshold = 120` is observable in machine-readable form, comparable across versions, and meaningful without prose interpretation.

It satisfies audit and review because the typed parameter changes can be recorded in audit trails that conform to 21 CFR Part 11 §11.10(e). Each modification has a discrete parameter, a discrete prior value, a discrete new value, an authoring identity, an authorizing identity, and a timestamp, none of which is obscured by subsequent modifications.

These properties are intrinsic to the primitive. They do not require external scanning, lockfiles, content-hash verification, or quarantine. The retrofitting effort that file-based ecosystems are now investing in is, in the parametric framing, redundant.

7.2 Connection to Established Operational Technology Primitives

The parametric model is not a novel proposal in the OT context. It is the established model.

OSIsoft and AVEVA's PI System [6] manages industrial time-series data through PI Points, each of which is a uniquely-named time-series stream with more than fifty typed attributes including point type, point source, compression deviation, archiving flag, and units of measure. PI Asset Framework adds a typed object-relational meta-model with strict units-of-measure classes. The format is parametric by design; a free-text procedural attribute does not exist in the data layer.

IEC 61131-3 and IEC 61499 specify function-block programming for industrial controllers. Function blocks are typed; instances have parameter sets, not free-form text. Ladder logic and structured text are compiled artifacts subject to controller-level change detection through cyclic redundancy checks.

Manufacturing Execution Systems (Siemens Opcenter, Rockwell FactoryTalk Pharma Suite, AVEVA System Platform) maintain configuration through typed object models, recipe management per ISA-88 (S88), and electronic batch records. Computerised Maintenance Management Systems (IBM Maximo, SAP Plant Maintenance, AVEVA Asset Performance Management) use ISO 14224–compliant taxonomies of equipment classes, failure-mode dictionaries, and work-order templates with typed fields.

In all of these systems, configuration is parametric, change is workflow-driven, and audit is intrinsic to the data model. The argument of this paper is that agent definitions in OT contexts should adopt the same primitive as the systems they integrate with, rather than importing a primitive optimized for horizontal developer tooling.

7.3 Architectural Precedent in Software

The parametric pattern in software has a substantial precedent in declarative configuration. Kubernetes [24] adopted desired-state YAML with controller reconciliation as the platform's core primitive, with GitOps providing auditable change history. Terraform [17] adopted declarative HashiCorp Configuration Language with state files as the authoritative model. NixOS [31]

extended the model with content-addressable storage, identifying packages and configurations by their hash.

These adoptions were driven by the same security and audit arguments that apply in the present case: convergent reconciliation, idempotency, GitOps auditability, rollback by re-application of prior manifests. The introduction of KYAML in Kubernetes 1.34 [24] is a concession that even declarative YAML required additional discipline, and is therefore not an objection to the pattern but a refinement of it.

Content-addressable artifacts are the established mechanism for tamper detection and provenance: Git objects, OCI image digests [33], IPFS CIDs [38]. The Supply-chain Levels for Software Artifacts framework [46] defines verifiable provenance as in-toto attestations signed by build systems, required by US Executive Order 14028 [47] and the EU Cyber Resilience Act. CycloneDX 1.5 [42] introduced ML-BOM (machine-learning bill of materials) for AI components; the present author notes that AIBOM standards are emerging rather than ratified, and that parametric agent definitions are natively compatible with the AIBOM direction of travel without claiming compliance with a not-yet-final standard.

The formal-methods and programming-languages literature provides additional support. The "Parse, don't validate" principle [22] and the LangSec program [29] argue that input validation should produce a precise, typed representation at the system boundary, with subsequent processing operating on the typed representation rather than on raw input. The Microsoft Security Response Center [28] reported that approximately 70 percent of all Microsoft CVEs over twelve years were memory-safety bugs that strong typing would have eliminated as a class. The argument by analogy is that agent definitions, treated as typed contracts rather than free-form text, eliminate the prompt-injection and skill-poisoning attack class as a class.

8 Limitations and Concessions

Six concessions are necessary for the argument to hold up to credible objection.

First, no production OT incident involving a file-based LLM skill is documented as of April 2026. The argument in section 6 is structural — the architectural class is being introduced into OT, the same class has produced documented exploits in IT — but it is not yet supported by an OT-specific incident. Section 9 argues that this absence is temporary and reflects penetration rather than safety.

Second, the parametric pattern has its own well-documented pathologies. The "YAML hell" critique [50] is real, as is the introduction of KYAML in Kubernetes 1.34 to address ambiguities in the YAML specification (the Norway Problem, indentation sensitivity, implicit type coercion). The configuration-language landscape has not converged: HashiCorp Configuration Language, JSON, TOML, Dhall, CUE, Jsonnet, Nickel, and Pkl all coexist. An organization adopting a parametric agent-definition pattern inherits a tooling-ecosystem decision. The right framing is "schemas plus content hashes plus provenance ledgers plus change-management workflow," not "any structured format is safer than any markdown."

Third, file-based agent definitions retain legitimate advantages. Procedural expressiveness is real: a 300-line incident-response playbook with branching narrative is awkward to encode as parameters. Low-ceremony authoring lowers the barrier for non-technical contributors; Anthropic's own SKILL.md format documentation [3] explicitly designs for users from non-technical backgrounds. GitHub-native distribution provides a community-contribution pattern that parametric registries have not yet matched. In horizontal developer tooling, these advantages are decisive.

Fourth, hybrid architectures are credible and should be acknowledged rather than dismissed. The pattern documented by Huang [18] — typed YAML frontmatter with name, description, version, platform constraints, and prerequisites, plus a markdown body, plus a content-hashed lockfile — is a credible middle ground. The argument of this paper is not that the markdown body must be eliminated, but that the parameters that determine the agent's operational envelope must be in the typed layer, not in the prose. Where a natural-language "intent" or "rationale" field is useful, it can be a typed field of a parametric wrapper rather than a free-form artifact.

Fifth, the Stuxnet attack [25] is a legitimate counterexample. The malware targeted parametric configuration — specifically, typed System Data Block values on Siemens S7-300 and S7-400 PLCs — rather than unstructured documents. The fair reading is that parametric systems reduce attack surface but do not eliminate it. The argument of this paper is that parametric systems raise the cost of attack, the certainty of detection, and the auditability of the result, not that they make attack impossible. The attacker who modified Stuxnet's PLC configurations did so through compromised engineering workstations and project files; the workstation file-system layer was not parametric, and was the entry point.

Sixth, the architectural commentary attributed to Huang [18] describes a specific implementation (Hermes Agent) that is documented in detail in the cited Substack chapter, but whose adoption beyond that single source has not been independently surveyed in this paper. The argument is supported by the pattern observable across multiple commercial offerings (Snyk–Tessl partnership, Cisco AI Defense, OWASP Agentic Skills draft) and does not depend on any single example. The structural claim — that the file-based primitive requires substantial external retrofit to provide properties parametric configurations possess by construction — holds regardless of the specific share of the market any one retrofit pattern has captured.

9 The Window of Inflection

The argument in sections 4 and 6 is structural and prospective. As of April 2026, the public record does not contain a documented OT incident involving a file-based LLM skill as the primary attack vector. The relevant question is not whether such incidents have occurred, but whether they will, and on what timescale.

LLM agents are entering OT environments through three vectors. The first is OEM-led integration: Yokogawa, Honeywell Forge, AVEVA, OSIsoft, Schneider Electric, Rockwell Automation, and Siemens have all announced AI agent capabilities or partnerships in the period 2024–2026. The second is operator-decision-support and advisory dashboards, in which agent recommendations are presented to human operators who retain control authority. The third is direct integration into Distributed Control Systems and historians, where agents may eventually have read-write access to control parameters.

Each of these vectors carries the file-based agent-definition primitive into OT contexts unless explicitly architected otherwise. The OEM offerings observed by the present author typically inherit Anthropic, OpenAI, or open-source skill specifications without modification. The default architectural choice is therefore the unstructured medium.

The penetration trajectory matters because the four-requirement convergence in section 6 is enforceable. A pharmaceutical manufacturer subject to 21 CFR Part 11, an electric utility subject to NERC CIP-010, a process-industry plant subject to OSHA PSM, and a high-risk AI system subject to EU AI Act Annex IV cannot, under audit, reconcile a markdown skill file with the regulatory expectation of unique identification, change control, status accounting, and audit. The reconciliation requires either (a) substantial additional engineering — content-hash

lockfiles, change-control workflows, scanner pipelines, and provenance ledgers wrapped around the markdown artifact — or (b) a primitive that satisfies the requirements by construction.

The choice point is now. Industrial AI deployments configured in 2026 will determine the architectural inheritance of OT agent systems for the remainder of the decade. The cost of changing the primitive after deployment is a multiple of the cost of choosing the primitive at deployment.

10 Recommendations

For original equipment manufacturers integrating LLM agents into industrial offerings: specify parametric agent-definition contracts in tender requirements and supplier specifications. Require that agent behavior be defined by typed, schema-validated parameter manifests with content hashes, and that natural-language fields, where retained, be discrete fields of typed wrappers rather than free-form artifacts. Require that change control operate at parameter granularity, with audit trails conforming to 21 CFR Part 11 §11.10(e) regardless of regulatory jurisdiction.

For regulators: issue clarifying guidance on the application of existing configuration-management, audit-trail, and Management-of-Change requirements to AI agents. The four-requirement convergence in section 6 already covers the relevant ground in principle; explicit guidance on its application to agent-definition artifacts would resolve the ambiguity that currently allows file-based skills to enter regulated environments under permissive readings of validation and document-control regimes.

For practitioners deploying LLM agents in or near regulated industrial processes: at minimum, treat skill files and agent definitions as configuration items under existing change-control regimes. Apply Management of Change to skill modifications. Require electronic signatures on changes to safety-related agent capabilities. Where parametric alternatives are available, prefer them for any agent whose behavior intersects with safety, quality, or regulated process variables.

For standards bodies — IEC, ISA, ISPE, NERC, OSHA, and the EU AI Act notified-body system: include explicit treatment of AI agents in next revisions. The current framework was written before agent-definition formats existed as a category; the absence of explicit guidance creates a vacuum that is being filled by horizontal tooling conventions optimized for different objectives. The four-requirement convergence in section 6 provides a starting point: any agent-definition artifact relied upon by a regulated process must satisfy unique identification, change control, status accounting, and audit, by intrinsic structure rather than by external retrofit.

11 Conclusion

The convergence of horizontal AI platforms on file-based agent-definition formats is now an established industry pattern. The pattern was optimized for low-friction authoring and rapid community contribution in horizontal developer tooling, and serves those objectives well. In the regulated industrial contexts that constitute operational technology — where IEC 62443, NERC CIP, 21 CFR Part 11, GAMP 5, IEC 61511, and EU AI Act Annex IV impose convergent requirements for unique identification, change control, status accounting, and audit — the pattern produces a structural mismatch with assurance objectives.

The mismatch is not one of safety in principle: file-based systems can be made to satisfy the four control objectives through external retrofit. The mismatch is one of cost. The retrofitting effort observable in 2025–2026 — content-hash lockfiles, scanner pipelines, trust-

tier marketplaces, install-policy matrices, atomic-write-and-rollback systems, and quarantine pipelines — represents a substantial and ongoing engineering investment to provide externally what parametric configurations provide intrinsically. In environments where the assurance cost is regulated rather than discretionary, that investment is unfavourable.

Empirical evidence from 2024–2026 establishes the file-based primitive as a production attack surface in IT and developer-tooling contexts. The architectural class is now migrating into OT through OEM integrations and operator-decision-support deployments. The window for selecting an appropriate primitive — before the file-based pattern becomes the default architectural inheritance of industrial AI for the remainder of the decade — is narrowing.

The argument of this paper is not that file-based agent definitions have failed. It is that they were optimized for objectives different from those imposed by industrial assurance regimes, and that imposing them on industrial contexts substitutes a discretionary primitive for a regulated one. Parametric agent definitions, defined as typed, schema-validated parameter manifests with deterministic content hashes, are the primitive the regulatory framework was written for. The choice between primitives in OT is a choice that should be made deliberately and at the architectural layer, rather than by default and at the deployment layer.

The horizontal AI vendors are designing for a world where the worst case is data leakage. The OT operators are responsible for a world where the worst case is loss of life. The architectural primitive optimized for the first will not survive contact with the second.

References

[1] Agent Skills Working Group. Agent Skills Open Standard. agentskills.io, 2025. URL https://agentskills.io. Accessed April 2026.

[2] AGENTS.md Working Group. AGENTS.md Specification. Linux Foundation, 2025. URL https://agents.md. Accessed April 2026.

[3] Anthropic. Equipping agents for the real world with agent skills. Anthropic Engineering Blog, October 2025. URL https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills. Accessed April 2026.

[4] Anthropic. Disrupting the first reported AI-orchestrated cyber espionage campaign. Anthropic News, November 2025. URL https://www.anthropic.com/news/disrupting-AI-espionage. Accessed April 2026.

[5] Antiy Labs. ClawHavoc: Analysis of large-scale poisoning campaign targeting the OpenClaw skill market for AI agents. Technical bulletin, Antiy Labs, February 2026. URL https://www.antiy.net/p/clawhavoc-analysis-of-large-scale-poisoning-campaign-targeting-the-openclaw-skill-market-for-ai-agents/.

[6] AVEVA. *PI System Architecture Planning and Implementation Workbook*. AVEVA Group plc, 2025.

[7] Check Point Research. Caught in the hook: RCE and API token exfiltration through Claude Code project files (CVE-2025-59536, CVE-2026-21852). Technical report, Check Point Software Technologies, February 2026. URL https://research.checkpoint.com/2026/rce-and-api-token-exfiltration-through-claude-code-project-files-cve-2025-59536/.

[8] Anton Cherepanov and Robert Lipovský. Industroyer: Biggest threat to industrial control systems since Stuxnet. ESET WeLiveSecurity, June 2017. URL https://www.welivesecurity.com/2017/06/12/industroyer-biggest-threat-industrial-control-systems-since-stuxnet/.

[9] Jason D. Christopher. SANS 2024 state of ICS/OT cybersecurity survey. Technical report, SANS Institute, October 2024. URL https://www.sans.org/white-papers/sans-2024-state-ics-ot-cybersecurity/.

[10] Cybersecurity and Infrastructure Security Agency, U.S. Department of Energy, National Security Agency, and Federal Bureau of Investigation. APT cyber tools targeting ICS/SCADA devices (Joint Cybersecurity Advisory AA22-103A). CISA Cybersecurity Advisory, April 2022. URL https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-103a.

[11] Edoardo Debenedetti, Jonas Geiping, Wieland Brendel, and Aleksander Madry. AgentDojo: A dynamic environment to evaluate attacks and defenses for LLM agents. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2406.13352.

[12] Dragos. Eighth Annual OT Cybersecurity Year in Review: 2025 operational technology threat landscape. Technical report, Dragos, Inc., 2025. URL https://www.dragos.com/dragos-2025-ot-cybersecurity-report-a-year-in-review.

[13] European Union. Regulation (EU) 2024/1689 (EU AI Act), article 11 and annex iv, 2024. URL https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

[14] Food and Drug Law Institute. FDA data integrity enforcement trends and practical mitigation measures. *Food and Drug Law Institute*, April 2018. URL https://www.fdli.org/.

[15] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, 2023. URL https://arxiv.org/abs/2302.12173.

[16] Cobus Greyling. What is AGENTS.md? Cobus Greyling on Medium, January 2026. URL https://cobusgreyling.medium.com/what-is-agents-md-2846b586b116.

[17] HashiCorp. *Terraform Documentation*. HashiCorp Inc., 2025. URL https://developer.hashicorp.com/terraform/docs.

[18] Ken Huang. Chapter 12: The skill system pattern (Claude Code vs. Hermes Agent). Agentic AI Series, Substack, April 2026. URL https://kenhuangus.substack.com/p/chapter-12-the-skill-system-pattern.

[19] Information Security Media Group. Anthropic's Cowork shipped with known vulnerability. CUInfoSecurity (ISMG), 2026. URL https://www.cuinfosecurity.com/anthropics-cowork-shipped-known-vulnerability-a-30553.

[20] International Electrotechnical Commission. *IEC 62443-4-1:2018 — Security for Industrial Automation and Control Systems — Secure Product Development Lifecycle Requirements*. International Electrotechnical Commission, 2018. URL https://webstore.iec.ch/publication/33615.

[21] International Society for Pharmaceutical Engineering. *GAMP 5: A Risk-Based Approach to Compliant GxP Computerized Systems, Second Edition*. ISPE, 2022. URL https://ispe.org/publications/guidance-documents/gamp-5-guide-2nd-edition.

[22] Alexis King. Parse, don't validate. Lexi Lambda Blog, November 2019. URL https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate. Accessed April 2026.

[23] Koi Security. ClawHavoc: 341 malicious ClawedBot skills found by the bot they were targeting. Research bulletin, Koi Security, February 2026. URL https://www.koi.ai/blog/clawhavoc-341-malicious-clawedbot-skills-found-by-the-bot-they-were-targeting.

[24] Kubernetes Authors. *Kubernetes Documentation: Working with Kubernetes Objects — Declarative Configuration*. The Kubernetes Project, 2025. URL https://kubernetes.io/docs/concepts/overview/working-with-objects/. Accessed April 2026.

[25] Ralph Langner. Stuxnet: Dissecting a cyberwarfare weapon. *IEEE Security & Privacy*, 9 (3):49–51, 2011. doi: 10.1109/MSP.2011.67. URL https://ieeexplore.ieee.org/document/5772960.

[26] Zhiyuan Li et al. Towards secure agent skills: Architecture, threat taxonomy, and security analysis. *arXiv preprint arXiv:2604.02837*, 2026. URL https://arxiv.org/abs/2604.02837.

[27] Mandiant. Attackers deploy new ICS attack framework "TRITON" and cause operational disruption to critical infrastructure. Mandiant Threat Research, December 2017. URL https://cloud.google.com/blog/topics/threat-intelligence/attackers-deploy-new-ics-attack-framework-triton.

[28] Matt Miller. Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape. BlueHat Israel 2019, Microsoft Security Response Center, 2019. URL https://github.com/microsoft/MSRC-Security-Research/blob/master/presentations/2019_02_BlueHatIL/2019_01%20-%20BlueHatIL%20-%20Trends%2C%20challenge%2C%20and%20shifts%20in%20software%20vulnerability%20mitigation.pdf.

[29] Falcon Momot, Sergey Bratus, Sven M. Hallberg, and Meredith L. Patterson. The seven turrets of Babel: A taxonomy of LangSec errors and how to expunge them. In *Proceedings of the 2016 IEEE Cybersecurity Development Conference (SecDev '16)*, pages 45–52, 2016. URL https://langsec.org/papers/langsec-cwes-secdev2016.pdf.

[30] National Institute of Standards and Technology. Special Publication 800-82 Revision 3: Guide to operational technology (OT) security. Technical report, NIST, September 2023. URL https://csrc.nist.gov/pubs/sp/800/82/r3/final.

[31] NixOS Project. *NixOS Documentation*. The NixOS Project, 2025. URL https://nixos.org/learn.

[32] North American Electric Reliability Corporation. *CIP-010-4 — Cyber Security: Configuration Change Management and Vulnerability Assessments*. NERC, 2020. URL https://www.nerc.com/pa/Stand/Reliability%20Standards/CIP-010-4.pdf.

[33] Open Container Initiative. *Open Container Initiative Distribution Specification*. OCI, 2024. URL https://github.com/opencontainers/distribution-spec.

[34] OpenAI. Codex Agent Skills documentation. OpenAI Developers, December 2025. URL https://developers.openai.com/codex/skills. Accessed April 2026.

[35] Pillar Security. New vulnerability in GitHub Copilot and Cursor: How hackers can weaponize code agents through compromised rule files. Research bulletin, Pillar Security, March 2025. URL https://www.pillar.security/blog/new-vulnerability-in-github-copilot-and-cursor-how-hackers-can-weaponize-code-agents.

[36] PromptArmor. Data exfiltration from Slack AI via indirect prompt injection. PromptArmor Disclosures, August 2024. URL https://www.promptarmor.com/resources/data-exfiltration-from-slack-ai-via-indirect-prompt-injection.

[37] PromptArmor. Claude Cowork exfiltrates files. PromptArmor Disclosures, January 2026. URL https://www.promptarmor.com/resources/claude-cowork-exfiltrates-files.

[38] Protocol Labs. *IPFS Documentation: Content Identifiers (CIDs)*. Protocol Labs, 2024. URL https://docs.ipfs.tech/concepts/content-addressing/.

[39] J. Qu et al. Supply-chain poisoning attacks against LLM coding agent skill ecosystems. *arXiv preprint arXiv:2604.03081*, 2026. URL https://arxiv.org/abs/2604.03081.

[40] Snyk Labs. ToxicSkills: Prompt injection in 36% and 1,467 malicious payloads in a study of agent skills supply chain compromise. Technical report, Snyk, February 2026. URL https://snyk.io/blog/toxicskills-malicious-ai-agent-skills-clawhub/. Accessed April 2026.

[41] The Hacker News. New rules file backdoor attack lets hackers inject malicious code via AI code editors. The Hacker News, March 2025. URL https://thehackernews.com/2025/03/new-rules-file-backdoor-attack-lets.html.

[42] The OWASP Foundation. *CycloneDX 1.5 Specification: Machine Learning Bill of Materials*. OWASP, 2023. URL https://cyclonedx.org/specification/overview/.

[43] The OWASP Foundation. *Top 10 for Agentic Applications 2026*. OWASP GenAI Security Project, 2026. URL https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/. Released 10 December 2025.

[44] The OWASP Foundation. Agentic Skills Top 10 (project page). OWASP, 2026. URL https://owasp.org/www-project-agentic-skills-top-10/. Accessed April 2026.

[45] The Register. Slack AI can leak private data via prompt injection. The Register, August 2024. URL https://www.theregister.com/2024/08/21/slack_ai_prompt_injection/.

[46] The SLSA Project. *Supply-chain Levels for Software Artifacts Specification, Version 1.1*. SLSA, 2024. URL https://slsa.dev/spec/v1.1. Accessed April 2026.

[47] U.S. Executive Office of the President. Executive Order 14028: Improving the nation's cybersecurity, May 2021. URL https://www.whitehouse.gov/briefing-room/presidential-actions/2021/05/12/executive-order-on-improving-the-nations-cybersecurity/.

[48] U.S. Food and Drug Administration. Guidance for industry: Part 11, electronic records; electronic signatures — scope and application, 2003. URL https://www.fda.gov/regulatory-information/search-fda-guidance-documents/part-11-electronic-records-electronic-signatures-scope-and-application.

[49] U.S. Occupational Safety and Health Administration. *Standard Interpretation: Management of Organizational Change*. OSHA, March 2009. URL https://www.osha.gov/laws-regs/standardinterpretations/2009-03-31.

[50] Ruud van Asseldonk. The YAML document from hell. Personal Blog, 2023. URL https://ruudvanasseldonk.com/2023/01/11/the-yaml-document-from-hell.

[51] Theodore J. Williams. *The Purdue Enterprise Reference Architecture: A Technical Guide for CIM Planning and Implementation*. Instrument Society of America, Research Triangle Park, NC, 1992.

[52] Simon Willison. OpenAI are quietly adopting skills, now available in ChatGPT and Codex CLI. Simon Willison's Weblog, December 2025. URL https://simonwillison.net/2025/Dec/12/openai-skills/.

About XMPro

XMPro is an industrial AI company headquartered in Dallas, Texas, building Multi-Agent Generative Systems (MAGS) for safety-critical operational technology environments. MAGS implements the parametric agent-definition pattern described in this whitepaper: typed schema-validated manifests, deterministic content identity, change control operating at parameter granularity, and audit trails designed to align with 21 CFR Part 11 §11.10(e) requirements.

XMPro works with operators in mining, oil and gas, utilities, and manufacturing across North America, Europe, the Middle East, and Australia. Its products integrate with the established operational-technology estate, including AVEVA PI System, Siemens, Rockwell, and major Distributed Control System and Supervisory Control and Data Acquisition platforms.

Contact

For more information, or to discuss the architectural questions raised in this paper:

- Pieter van Schalkwyk, CEO — pieter.vanschalkwyk@xmpro.com
- Gavin Green, Chief Solutions Officer — gavin.green@xmpro.com
- Web: xmpro.com

© 2026 XMPro Inc. All rights reserved.